

Protótipo para detecção, localização e seleção automática de ação por sensoramento

Modelo de Banco de Dados

Gabriel Vasconcellos de Moraes — Matheus Emanuel Gomes Souza

Orientador: Anker Douglas Loss

Curso: Engenharia da Computação — Faculdade do Centro Leste (UCL)

1 Apresentação do Modelo de Dados

O RadarTorres é um protótipo que detecta alvos por sensoramento, seleciona automaticamente a torre mais adequada e permite o acionamento manual ou automático de um indicador demonstrativo. Para cumprir esse papel, o sistema precisa preservar, entre execuções, dados de naturezas distintas: contas de usuário e seus perfis de acesso, o histórico de objetos detectados pelo radar, a auditoria de toda ação de acionamento e de toda troca de modo de operação, e os chamados de suporte abertos pelos operadores.

Além dos dados de auditoria, o sistema mantém preferências de interface por usuário (idioma, tema, tela inicial), o layout personalizado dos painéis (posição, tamanho e visibilidade de cada card), as zonas mortas configuradas por um administrador e as configurações de ambiente do Arduino (porta serial, placa, sketch). Cada grupo tem finalidade e ciclo de vida próprios, mas todos precisam sobreviver ao fechamento da aplicação.

Por não depender de um servidor de banco de dados, o protótipo prioriza simplicidade de instalação e execução local/offline — coerente com o estágio atual do projeto (protótipo acadêmico, uso demonstrativo). As seções seguintes descrevem a tecnologia de persistência empregada, suas vantagens e limitações, as principais entidades identificadas no código e o modelo proposto para uma futura migração relacional.

2 Tecnologia e Implementação

O protótipo atual **não utiliza um Sistema Gerenciador de Banco de Dados relacional**. A persistência é realizada localmente por meio de arquivos CSV e JSON, sem servidor e sem instalação adicional — portanto não há uma *versão de SGBD* a declarar.

Seis entidades usam arquivos **CSV**, um por tabela, em %AppData%\RadarTorres\Data\ (pasta do usuário do Windows, sem privilégio de administrador) — ver a coluna "Persistência" da Tabela 2. Cada arquivo é criado com cabeçalho na primeira execução por `CsvTableStore<T>`; não há *migration* tradicional — o próprio código (modelo + mapeamento de colunas) define o schema.

Outras três entidades usam arquivos **JSON** individuais em %LocalAppData%\RadarTorres\ : um único arquivo por instalação para zonas mortas e para configurações do Arduino, e um arquivo por tela e por usuário para o layout dos painéis (nome com o padrão `dashboard-layout-{tela}-{usuário}.json`). Cada entidade é acessada exclusivamente por uma interface de repositório dedicada (ex.: `IUsuarioRepository`, `IDeadZoneRepository`), isolando `ViewModels` e `Services` do formato de armazenamento.

Implementação futura proposta: existe um plano documentado de migração para um banco relacional (SQLite ou Entity Framework Core), trocando apenas a implementação de cada repositório sem alterar `ViewModels/Services`. Essa tecnologia *ainda não foi implementada* — não há arquivo de banco, tabela SQL ou *migration* no repositório hoje.

3 Vantagens e Limitações da Solução Atual

Tabela 1: Vantagens e limitações da persistência atual (CSV/JSON)

Vantagens	Limitações
Simplicidade de instalação, sem servidor	Sem integridade referencial imposta pelo armazenamento
Funcionamento local/offline	Consultas limitadas (sem SQL; filtragem em memória)
Baixo acoplamento via interfaces de repositório	Tende a não escalar bem para grande volume de dados
Fácil distribuição (basta copiar a pasta de dados)	Sem controle de concorrência/transações reais
Adequado ao estágio atual de protótipo	Relacionamentos garantidos só pelo código da aplicação

Essas limitações não indicam uma decisão equivocada: CSV/JSON atendem plenamente ao escopo de um protótipo de uso demonstrativo. A Seção 6 apresenta o modelo pensado para quando a evolução do projeto exigir um SGBD relacional.

4 Principais Dados do Sistema

Tabela 2: Entidades persistidas identificadas no código-fonte

Entidade	Finalidade	Persistência
Usuario	Conta de acesso, perfil e autenticação	CSV usuarios.csv
PreferenciasUsuario	Preferências de interface por usuário	CSV preferencias_usuario.csv
ObjetoDetectado	Histórico de detecções do radar	CSV objetos_detectados.csv
AcaoRealizada	Auditoria de acionamentos (só inserção)	CSV acoes_realizadas.csv
AlteracaoModo	Auditoria de troca de modo de operação	CSV alteracoes_modos.csv
ChamadoAjuda	Chamados de suporte dos usuários	CSV chamados_ajuda.csv
DeadZone	Áreas onde alvos são ignorados pelo sistema	JSON dead-zones.json
Dashboard CardLayout	Layout dos cards do painel, por tela/usuário	JSON dashboard-layout-*.json
ArduinoSettings	Configurações de ambiente do Arduino	JSON arduino-settings.json

5 Modelo e Relacionamentos

A Figura 1 apresenta as entidades de usuários e configurações; a Figura 2, as de monitoramento e auditoria; e a Figura 3, as de segurança e suporte. *Usuario* aparece nas três, como referência dos relacionamentos que partem dele.

Usuario se relaciona 1:1 com PreferenciasUsuario (chave UsuarioId, igual a Usuario.Id) e 1:N com AcaoRealizada e AlteracaoModo — nesses dois últimos casos por **Login** (texto), sem chave estrangeira real — e com ChamadoAjuda, por Id numérico. DashboardCardLayout se associa a Usuario apenas pelo *nome do arquivo* (tela-idUsuario); é uma convenção de nomenclatura, não uma chave estrangeira. ObjetoDetectado **não possui nenhum vínculo persistido** com AcaoRealizada, Usuario ou qualquer outra entidade — são registros independentes. DeadZone e ArduinoSettings também não se relacionam com nenhuma outra entidade: são configurações únicas por instalação, não por usuário.

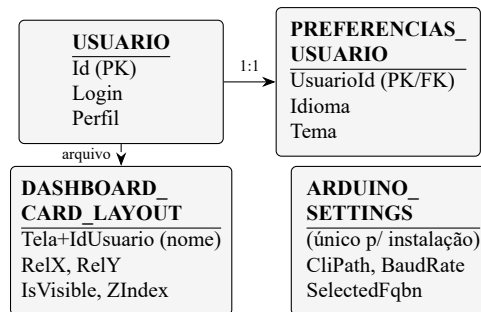


Figura 1: Modelo de dados relacionado aos usuários e às configurações. ArduinoSettings não possui relação com nenhuma outra entidade (arquivo único da instalação).

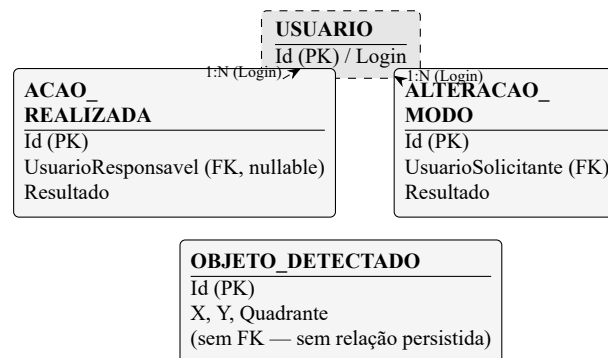


Figura 2: Modelo de dados relacionado ao monitoramento e à auditoria. ObjetoDetectado não possui vínculo persistido com nenhuma outra entidade.

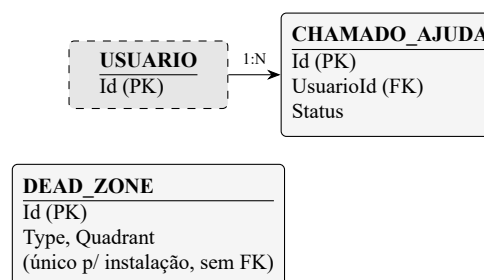


Figura 3: Modelo de dados relacionado à segurança e ao suporte. DeadZone não possui relação com nenhuma outra entidade.

6 Modelo Relacional Proposto

Esta seção descreve uma proposta de migração futura, não o banco atual — nenhuma dessas tabelas, chaves ou *migrations* existe hoje no repositório. A estrutura de dados seria a mesma, mas as referências

textuais por Login passariam a ser chaves estrangeiras íntegras por Id numérico, com integridade garantida pelo próprio SGBD (Figura 4).

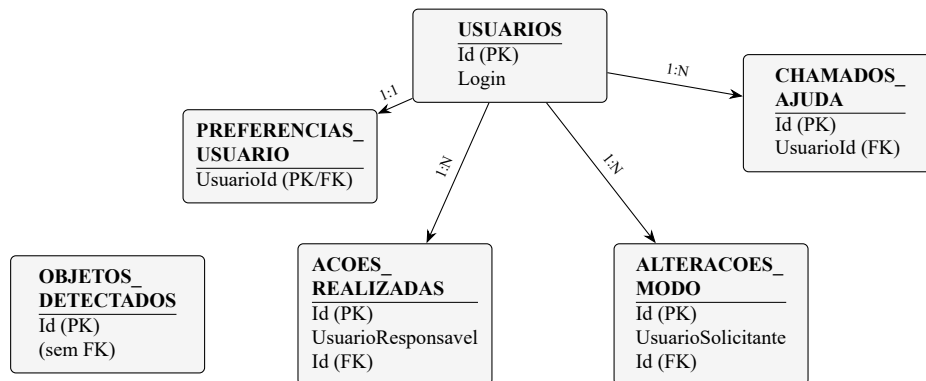


Figura 4: Modelo relacional proposto para a migração futura de persistência (CSV/JSON → SQL). *ObjetosDetectados* permanece sem chave estrangeira.

7 Conclusão

A persistência atual, baseada em arquivos CSV e JSON, atende plenamente ao estágio de protótipo do RadarTorres, mantendo a instalação simples e o funcionamento totalmente local. O uso de uma interface de repositório dedicada para cada entidade mantém a camada de dados desacoplada das demais camadas da aplicação, o que viabiliza uma futura migração para um banco relacional de forma incremental, sem exigir mudanças em ViewModels ou Services. O modelo atual já preserva as informações necessárias de operação, auditoria e configuração exigidas pelo sistema.

REFERÊNCIAS

[REFERÊNCIA BIBLIOGRÁFICA A INSERIR]